

Data Analytics Capstone
Predicting Service Level Agreement Breach in IT Incident Management: An Analysis of
Ticket-Level Drivers in an Enterprise ServiceNow Event Log
Synopsis

Binesh Balakrishnan
Walsh College
QM640: Data Analytics Capstone
Ms Keya Choudhury Ganguli
Term 3
October 6, 2026

Table of Contents

Introduction.....3

 Background and Context.....3

 Problem Statement.....4

 Purpose of the Study.....4

 Significance of the Study.....5

Scope and Objectives.....5

 Research Questions and Hypotheses.....6

Sample Size Calculation.....7

Data Description.....10

 Source and Access.....10

 Data Ethics, Privacy, and Bias.....11

 Structure of the Data.....12

 Data Preparation and Preliminary Profile.....15

Analytic Approach.....16

 Solution to RQ1: Priority and SLA Breach.....17

 Solution to RQ2: Reassignment and SLA Breach.....17

 Solution to RQ3: Knowledge-Base Use and SLA Breach.....18

 Solution to RQ4: Predicting SLA Breach at Logging.....19

 End-to-End Solution Pipeline.....19

 Limitations and Threats to Validity.....21

Recommendation and Application.....21

Project Plan and Resources.....22

References.....24

Appendix A.....27

Introduction

Background and Context

Incident management is the IT service management (ITSM) process that restores normal service as quickly as possible after an unplanned interruption. In enterprise environments it is usually run on a ticketing platform such as ServiceNow, where each incident is logged, prioritised from its impact and urgency, routed to a support group, worked, resolved, and closed. Frameworks such as the IT Infrastructure Library (ITIL) formalise this lifecycle, and organisations adopting them report gains in service quality but persistent difficulty in measuring and improving process performance (Marrone & Kolbe, 2011).

In managed services, the performance of this process is governed by a service level agreement (SLA): a contractual commitment to resolve incidents within target times that depend on priority. An SLA breach is therefore a commercial event as well as an operational one. Breaches erode customer confidence, can trigger service credits in outsourcing contracts, and consume the time of the service delivery managers who must explain them in monthly reviews. The stakeholders are the end users who experience the outage, the service desk and resolver groups who work the tickets, the service delivery manager accountable for SLA attainment, and the client who pays for the service.

Two operational patterns are widely recognised as drivers of delay: reassignment, where a ticket is passed between support groups before reaching one able to resolve it (Shao et al., 2008), and the use or non-use of recorded knowledge. The rise of artificial intelligence for IT operations (AIOps) has created interest in using historical operational data to predict and prevent service problems rather than react to them (Dang et al., 2019). The academic basis for such prediction is predictive process monitoring, which uses event logs to forecast the remaining time or outcome of running cases (van der Aalst, 2016; Tax et al., 2017; Verenich et al., 2019). The dataset used here has previously been used to study attribute selection for completion-time prediction (Amaral et al., 2018a, 2019), but not to test the managerial hypotheses that service

delivery managers act on, nor to frame SLA breach as a classification problem at the moment of logging. That gap motivates this study.

Problem Statement

In enterprise incident management, SLA attainment is monitored after the fact: breaches are counted at month end, and the ticket attributes and process behaviours that preceded them are rarely tested statistically. Service delivery managers therefore make routing and knowledge-management decisions on intuition, and at-risk tickets are escalated only once the SLA clock is nearly exhausted. The problem this study addresses is the absence of evidence on which ticket-level factors are significantly associated with SLA breach, and whether breach risk can be predicted at the point of logging more accurately than by priority alone. The study has a single dependent variable across all four research questions: **SLA breach** (binary, 1 = breached, 0 = met), taken from the event log. The independent variables are **priority** (four ordinal levels; RQ1), **reassignment count** (integer; RQ2), **knowledge-base use** (binary; RQ3), and the **attributes recorded at logging** (RQ4), with priority and service category used as controls in RQ2 and RQ3. Focusing on one outcome keeps every analysis aligned to the measure that service contracts are judged on.

Purpose of the Study

This research aims to identify the ticket-level drivers of SLA breach in an enterprise IT incident-management process, and to test whether a model built on attributes available at ticket logging can predict breach risk, in order to support earlier escalation and better assignment decisions in managed-services operations. The study is both explanatory (RQ1 to RQ3, hypothesis tests on drivers) and predictive (RQ4, binary classification). Specifically, it asks whether the breach rate differs by priority (RQ1), whether reassignment is associated with breach (RQ2), whether knowledge-base use is associated with breach (RQ3), and whether an at-logging model predicts breach better than a priority-only rule (RQ4).

Significance of the Study

For a managed-service provider, SLA attainment is reported to the client every month and shapes renewals, so knowing which ticket-level factors drive breaches lets managers target interventions where they matter rather than across the board. A risk score available when a ticket is logged would move escalation from reactive to proactive, giving resolver groups the full SLA window to act on high-risk tickets. Academically, the study extends predictive process monitoring on this log from completion-time regression to managerially testable hypotheses and to breach classification at logging, evaluated against the simple rule that practitioners already use.

Scope and Objectives

The study uses a single, publicly available event log extracted from the ServiceNow audit system of a real IT company (Amaral et al., 2018b). The unit of analysis is the incident: the 141,712 events are collapsed to 24,918 incidents so that each incident contributes one observation. The objectives are to (a) quantify how the SLA breach rate differs across priority levels, (b) estimate the association between reassignment and SLA breach, (c) estimate the association between knowledge-base use and SLA breach, and (d) build and evaluate a breach-prediction model that uses only information available at logging.

Boundaries. In scope are the four research questions, descriptive profiling, and a validated prototype model, all on this dataset. Out of scope are the design of SLA targets (which are not recorded), staffing or roster optimisation, causal claims about interventions, real-time deployment into a ticketing system, and text analytics (the log has no free-text fields). Resolution time is used only to describe the data, not as an outcome, which keeps the analysis focused on one dependent variable. This keeps the scope broad enough to combine three inferential analyses with a comparative machine-learning study, but feasible on one dataset of 24,918 incidents in 10 weeks with open-source tools. To protect validity, RQ4 excludes variables that accumulate during the ticket lifecycle, such as reassignment count, reopen count, and update count, because they are not known at logging and would leak future information into the prediction.

Research Questions and Hypotheses

The four research questions are facets of the single business problem of SLA attainment and are all answered from the same incident-level dataset. Each is paired with a null (H_0) and an alternative (H_1) hypothesis, tested at $\alpha = 0.05$, and Table 1 summarises the dependent variable, independent variables, and primary test for each. All four share the same dependent variable, SLA breach.

Research Question 1 (RQ1)

Does the SLA breach rate differ significantly across the four priority levels?

H_{0_1} : SLA breach is independent of priority; the breach rate is the same for priority levels 1 (Critical) to 4 (Low).

H_{1_1} : SLA breach is associated with priority; the breach rate differs for at least one priority level.

Research Question 2 (RQ2)

Is the number of reassignments associated with the likelihood of SLA breach, after controlling for priority and service category?

H_{0_2} : Reassignment is not associated with SLA breach (odds ratio for reassignment count = 1; $\beta_1 = 0$).

H_{1_2} : Reassignment is associated with SLA breach (odds ratio $\neq 1$; $\beta_1 \neq 0$).

Research Question 3 (RQ3)

Is knowledge-base use associated with the likelihood of SLA breach, after controlling for priority and service category?

H_{0_3} : Knowledge-base use is not associated with SLA breach (odds ratio for knowledge use = 1; $\beta_1 = 0$).

H_{1_3} : Knowledge-base use is associated with SLA breach (odds ratio $\neq 1$; $\beta_1 \neq 0$).

Research Question 4 (RQ4)

Can a machine-learning classifier using only attributes available at ticket logging predict SLA breach with a significantly higher area under the ROC curve (AUC) than a priority-only baseline model?

H0₄: AUC of the best machine-learning model = AUC of the priority-only logistic regression baseline.

H1₄: AUC of the best machine-learning model > AUC of the priority-only baseline.

Table 1

Research Questions, Variables, and Primary Tests

RQ	Dependent variable	Independent variable(s)	Primary test / model
RQ1	SLA breach (0/1)	Priority (4 levels)	χ^2 test of independence (4×2); pairwise χ^2 tests with Holm correction
RQ2	SLA breach (0/1)	Reassignment count; controls: priority, category	χ^2 test of independence; multivariable logistic regression (Wald test)
RQ3	SLA breach (0/1)	Knowledge-base used (0/1); controls: priority, category	χ^2 test of independence; multivariable logistic regression (Wald test)
RQ4	SLA breach (0/1)	At-logging attributes only (Table 5)	Logistic regression, random forest, XGBoost; DeLong test on AUC

Note. SLA = service level agreement; AUC = area under the receiver operating characteristic curve.

Sample Size Calculation

Sample sizes were calculated a priori for each research question, choosing for each the method that matches its planned analysis (Table 2). All calculations use $\alpha = 0.05$ (two-tailed) and power $(1 - \beta) = 0.80$, so that the true effect must lie 2.8 standard errors from the null value (1.96 + 0.84; Gelman & Hill, 2007). Effect sizes are Cohen's (2013) "small" conventions, so the study is powered for modest but operationally meaningful effects. Three parameters were taken from the data rather than assumed: the overlap of reassignment (RQ2) and knowledge-base use (RQ3) with the control variables, and the breach prevalence (RQ4). Calculations follow G*Power procedures (Faul et al., 2007) and were reproduced in Python (Appendix A). Table 3 shows each calculation step by step.

Table 2*Sample Size Method, Justification, and Parameters by Research Question*

RQ	Method	Justification (alignment with analysis)	Parameters
RQ1	Power analysis: χ^2 test of independence (noncentral χ^2)	RQ1 tests whether a binary outcome (breach) is independent of a 4-level factor (priority), which is a 4×2 contingency table with $df = (4 - 1)(2 - 1) = 3$	$\alpha = 0.05$; power = 0.80; $w = 0.10$ (small); $df = 3$
RQ2	Power analysis: χ^2 test of independence (noncentral χ^2), with covariate adjustment	H_{02} is first tested on a 2×2 table (reassigned $\times$ breached); the logistic model adds covariates, so the requirement is inflated by $1 / (1 - R^2)$ of reassignment on those covariates	$\alpha = 0.05$; power = 0.80; $w = 0.10$ (small); $df = 1$; $R^2 = 0.047$ (observed)
RQ3	Power analysis: χ^2 test of independence (noncentral χ^2), with covariate adjustment	H_{03} is first tested on a 2×2 table (knowledge used $\times$ breached); the logistic model adds priority and category, so the requirement is inflated by $1 / (1 - R^2)$ of knowledge use on those covariates	$\alpha = 0.05$; power = 0.80; $w = 0.10$ (small); $df = 1$; $R^2 = 0.154$ (observed)
RQ4	Confidence-interval method: precision of AUC (Hanley & McNeil, 1982)	RQ4 compares AUCs; DeLong's test depends on the standard error of each AUC on the test set, so the test set is sized to estimate AUC within ± 0.03	AUC = 0.75; 95% CI half-width $e = 0.03$; prevalence $p = 0.366$ (observed); 30% hold-out

Note. Effect-size conventions from Cohen (2013). Observed values are from the incident-level data (see Table 6).

Table 3*Stepwise Sample Size Calculations*

RQ	Step and formula	Substitution	Result
RQ1	1. Noncentrality needed for $df = 3$, $\alpha = 0.05$, power = 0.80	λ from the noncentral χ^2 distribution, $df = 3$	$\lambda = 10.903$
	2. $N_{\chi^2} = \lambda / w^2$	$10.903 / 0.10^2 = 1,090.3$	1,091
RQ2	1. Noncentrality needed for $df = 1$, $\alpha = 0.05$, power = 0.80	$\lambda = (1.960 + 0.842)^2$	$\lambda = 7.849$
	2. $N_{\chi^2} = \lambda / w^2$	$7.849 / 0.10^2 = 784.9$	785
	3. $N_{adj} = N_{\chi^2} / (1 - R^2)$	$785 / (1 - 0.047) = 823.7$	824
RQ3	1. Noncentrality needed for $df = 1$, $\alpha = 0.05$, power = 0.80	$\lambda = (1.960 + 0.842)^2$	$\lambda = 7.849$
	2. $N_{\chi^2} = \lambda / w^2$	$7.849 / 0.10^2 = 784.9$	785
	3. $N_{adj} = N_{\chi^2} / (1 - R^2)$	$785 / (1 - 0.154) = 927.9$	928
RQ4	1. $Q_1 = A / (2 - A)$; $Q_2 = 2A^2 / (1 + A)$	$0.75 / 1.25$; $2(0.5625) / 1.75$	$Q_1 = 0.600$; $Q_2 = 0.643$

RQ	Step and formula	Substitution	Result
	2. $SE^2 = [A(1 - A) + (n_p - 1)(Q_1 - A^2) + (n_n - 1)(Q_2 - A^2)] / (n_p + n_n)$; smallest test set with $1.96 \times SE \leq 0.03$	$n_{test} = 1,191$; $n_p = 436$ breaches, $n_n = 755$; $SE^2 = 77.089 / 329,180 = 0.000234$; $SE = 0.0153$	$1.96 \times 0.0153 = 0.0300$ ✓
	3. $N_{total} = n_{test} / 0.30$	$1,191 / 0.30$	3,970

Note. λ = noncentrality parameter; R^2 = share of the variance in the independent variable explained by priority and category; A = assumed AUC; n_p and n_n = breached and non-breached incidents in the test set. Results are rounded up to the next whole incident. AUC variance formula for RQ4 from Hanley and McNeil (1982).

Two refinements check that the calculations fit the analyses. For RQ1, the priority groups are unequal (270 Critical, 408 High, 23,466 Moderate, and 774 Low incidents), but Cohen's w already reflects the marginal proportions, and the smallest expected cell count in the 4×2 table is 98.8, well above the minimum of 5 that the χ^2 test requires. For RQ4, the hypothesis compares two AUCs scored on the same incidents; with each AUC estimated to $SE = 0.0153$ and an assumed correlation of 0.5 between the two models' scores, the standard error of the difference is $\sqrt{(2 \times 0.0153^2 \times 0.5)} = 0.0153$, so a difference of $2.8 \times 0.0153 = 0.043$ in AUC is detectable with 80% power.

The assumptions are consistent across the four questions: every calculation uses $\alpha = 0.05$, 80% power or 95% confidence, and Cohen's small-effect conventions, and the data-derived inputs ($R^2 = 0.047$ and 0.154 , and prevalence 0.366) come from the same incident-level file used in the analysis. The confidence-interval method is used for RQ4 rather than a classical power formula because DeLong's statistic is built from the standard errors of the two AUCs, so controlling those standard errors controls the test; using a two-sided 95% interval is conservative relative to the one-sided alternative H_{14} . For the multivariable logistic models in RQ2 and RQ3, Green's (1991) rule of $N \geq 50 + 8m$ with $m = 10$ predictors gives $N \geq 130$, below the 824 and 928 required, so the χ^2 -based calculations are binding. Table 4 summarises the method, key parameters, and minimum sample size for each research question.

Table 4*Sample Size Summary and Final Sample Size*

Research Question	Method Used	Key Parameters	Minimum Sample Size (N)
RQ1	Power (χ^2 test of independence, 4×2)	$\alpha = 0.05$, Power = 0.80, Effect size $w = 0.10$, $df = 3$	1,091
RQ2	Power (χ^2 test of independence, covariate-adjusted)	$\alpha = 0.05$, Power = 0.80, Effect size $w = 0.10$, $df = 1$, $R^2 = 0.047$	824
RQ3	Power (χ^2 test of independence, covariate-adjusted)	$\alpha = 0.05$, Power = 0.80, Effect size $w = 0.10$, $df = 1$, $R^2 = 0.154$	928
RQ4	Confidence interval (AUC precision; Hanley & McNeil, 1982)	$Z = 1.96$ (95% CI), $e = 0.03$, AUC = 0.75, $p = 0.366$, 30% hold-out	3,970
Final N	Maximum of RQ1–RQ4 minimums	Available: 24,918 incidents ($6.3 \times$ final N)	3,970

Note. Available N is 24,918 for all four research questions, because every incident has a recorded SLA outcome.

Final sample size decision. Following the rule that the final sample size is the maximum of the minimum sample sizes across research questions, $N = \max(1,091; 824; 928; 3,970) = 3,970$ incidents, driven by RQ4. Because the dataset contains 24,918 incidents, 6.3 times this requirement, the study will use all available incidents rather than a sample. This exceeds every minimum, gives higher power than required for RQ1 to RQ3, and leaves a temporal test set of about 7,475 incidents for RQ4, 6.3 times the 1,191 required.

Data Description

Source and Access

The data is the Incident Management Process Enriched Event Log, donated to the University of California, Irvine (UCI) Machine Learning Repository by Amaral et al. (2018b) and available at <https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log> (DOI 10.24432/C57S4H). It was extracted from the audit system of a ServiceNow instance used by an IT company and enriched with data from the platform's relational database. It is real operational

data: it is not synthetic and is not sourced from Kaggle. The complete data and code are published in the public GitHub repository

https://github.com/binesh29/QM640_data_analytics_capstone, organised as shown in Figure 1.

The README states the data source and licence and gives the commands to install the requirements and run the notebooks.

Figure 1

Structure of the GitHub Repository

QM640_data_analytics_capstone/	
├ README.md	<i>Project summary, data source, how to run</i>
├ requirements.txt	<i>Pinned Python package versions</i>
├ data/	
│ └ raw/incident_event_log.csv	<i>UCI source file, unchanged (141,712 rows)</i>
│ └ processed/incidents.csv	<i>One row per incident (24,918 rows)</i>
├ notebooks/	
│ └ 01_data_preparation.ipynb	<i>Load, validate, collapse, derive, profile</i>
├ src/	
│ └ sample_size.py	<i>Reproduces Tables 3 and 4</i>
└ docs/	
└ └ Synopsis (Word and PDF)	

Note. Notebooks 02 to 05 (one per research question) will be added after the synopsis is approved.

Data Ethics, Privacy, and Bias

The dataset is released under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence, which permits reuse and redistribution with attribution. Personal and organisational identifiers such as caller, analyst, location, and support group are replaced by the donors with anonymised labels (for example, Caller 2403 or Group 56); the study involves no human participants and makes no attempt to re-identify individuals, and analyst-level identifiers are not used as predictors so that the model cannot be used to rate individual staff. Four sources of bias are addressed for the single outcome, SLA breach. Selection bias: only closed incidents from one organisation are included, so findings are framed as evidence from one enterprise rather than general laws. Temporal bias: 98.1% of incidents fall in March to May 2016, so the RQ4 split is

time-ordered to avoid optimistic estimates. Label bias: the `made_sla` flag reflects targets that are not recorded, so breach is analysed as defined by the organisation. Class imbalance: breaches are 36.6% of incidents and Critical and High priorities only 2.7%, so AUC, recall, and class weighting are used rather than accuracy alone.

Structure of the Data

The raw file is a single comma-separated table of 141,712 rows and 36 columns in which each row is an event, that is, the state of an incident after an update (Figure 2). Loading the file confirmed 24,918 distinct incidents, an average of 5.7 events per incident, and missing values coded as a question mark. The `incident_state` variable takes eight named levels plus an anomalous value of -100 on 5 rows, and every incident ends in the Closed state. Incidents were opened between 29 February 2016 and 16 February 2017, with 98.1% (24,437) opened between March and May 2016. Table 5 defines every source and derived variable.

Figure 2

Extract of the Raw Event Log (First Six Events, Eight of 36 Columns)

number	incident_state	reassignment_count	made_sla	opened_at	priority	knowledge	resolved_at
INC0000045	New	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Closed	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000047	New	0	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52
INC0000047	Active	1	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52

Note. Each incident (number) appears on several rows as it moves through states; the analysis keeps one row per incident. From *Incident Management Process Enriched Event Log* [Data set], by C. Amaral, M. Fantinato, and S. Peres, 2018, UCI Machine Learning Repository (<https://doi.org/10.24432/C57S4H>). CC BY 4.0.

Table 5

Data Dictionary

Variable	Type	Units / levels	Definition (UCI metadata)	Role in study
number	ID	Text code (24,918)	Incident identifier	Unit-of-analysis key
incident_state	Categorical	8 levels + "-100"	State of the incident at that event	Select final record

Variable	Type	Units / levels	Definition (UCI metadata)	Role in study
active	Boolean	true / false	Whether the record is active or closed	Data check
reassignment_count	Integer	Count, 0–27	Times the support group or analyst changed	IV (RQ2); excluded from RQ4
reopen_count	Integer	Count	Times the caller rejected the resolution	Descriptive; excluded from RQ4
sys_mod_count	Integer	Count	Number of updates to that moment	Event ordering; excluded from RQ4
made_sla	Boolean	true = SLA met	Whether the incident met its SLA target	Source of the DV (all RQs)
caller_id	ID	Label (5,244)	Affected user	Predictor (RQ4, frequency-encoded)
opened_by	ID	Label	User who reported the incident	Not used
opened_at	Datetime	dd/mm/yyyy hh:mm	Incident opening time	Start time; hour/day features; split order
sys_created_by	ID	Label	User who registered the incident	Not used
sys_created_at	Datetime	dd/mm/yyyy hh:mm	System creation time (37.5% missing)	Data check
sys_updated_by	ID	Label	User who made the update	Not used
sys_updated_at	Datetime	dd/mm/yyyy hh:mm	System update time	Event ordering
contact_type	Categorical	5 levels (99.1% phone)	Reporting channel	Predictor (RQ4)
location	ID	Label (224)	Affected location	Predictor (RQ4, encoded)
category	Categorical	58 levels	First-level affected service	Control (RQ2, RQ3); predictor (RQ4)
subcategory	Categorical	254 levels	Second-level affected service	Predictor (RQ4)
u_symptom	Categorical	525 levels	User perception of availability	Predictor (RQ4)
cmdb_ci	ID	Label (99.7% missing)	Affected configuration item	Missingness flag (RQ4)
impact	Ordinal	1 High, 2 Medium, 3 Low	Impact of the incident	Predictor (RQ4)
urgency	Ordinal	1 High, 2 Medium, 3 Low	Urgency stated by the user	Predictor (RQ4)

Variable	Type	Units / levels	Definition (UCI metadata)	Role in study
priority	Ordinal	1 Critical – 4 Low	Calculated from impact and urgency	IV (RQ1); control (RQ2, RQ3); baseline (RQ4)
assignment_group	ID	Label (78)	Support group in charge	Predictor (RQ4, initial group)
assigned_to	ID	Label	Analyst in charge	Not used (privacy)
knowledge	Boolean	true / false	Whether a knowledge-base document was used	IV (RQ3); RQ4 sensitivity only
u_priority_confirmation	Boolean	true / false	Whether priority was double-checked	Predictor (RQ4)
notify	Categorical	2 levels	Whether notifications were generated	Predictor (RQ4)
problem_id	ID	Label (98.4% missing)	Associated problem record	Descriptive
rfc	ID	Label (99.3% missing)	Associated change request	Descriptive
vendor	ID	Label (99.8% missing)	Vendor in charge	Descriptive
caused_by	ID	Label ($\approx 100\%$ missing)	Change that caused the incident	Not used
closed_code	Categorical	17 levels	Resolution code (close_code in UCI metadata)	Descriptive
resolved_by	ID	Label	User who resolved the incident	Not used (privacy)
resolved_at	Datetime	dd/mm/yyyy hh:mm	Time of user resolution	Descriptive (resolution_hours)
closed_at	Datetime	dd/mm/yyyy hh:mm	Time of closure	Descriptive
resolution_hours (derived)	Continuous	Hours, > 0	resolved_at – opened_at, final record	Descriptive only (not an outcome)
sla_breach (derived)	Binary	1 = breached, 0 = met	1 if made_sla is false on final record	DV (RQ1–RQ4)
reassigned (derived)	Binary	1 = ≥ 1 reassignment	1 if reassignment_count ≥ 1	IV (RQ2 χ^2 test)

Note. IV = independent variable; DV = dependent variable. Adapted from *Incident Management Process Enriched Event Log* [Data set], by C. Amaral, M. Fantinato, and S. Peres, 2018, UCI Machine Learning Repository (<https://doi.org/10.24432/C57S4H>). CC BY 4.0. Missing percentages are at event level. The UCI description of made_sla is ambiguous; the data confirms that true means the SLA was met (median resolution 1.1 hours when true, 201.7 hours when false).

Data Preparation and Preliminary Profile

Preparation follows five steps, implemented in the notebook 01_data_preparation.ipynb: (1) convert the missing-value code and parse timestamps; (2) sort events by incident and update count, and keep the first record (attributes known at logging, used for RQ4) and the final record (outcome and accumulated counts, used for RQ1 to RQ3); (3) derive the outcome `sla_breach` for all 24,918 incidents, plus resolution time for description only (1,556 incidents have no resolution timestamp and 39 have zero duration, which does not affect the outcome); (4) frequency-encode high-cardinality identifiers and group rare categories; and (5) profile missingness, outliers, and class balance.

The first descriptive pass (Table 6) shows that 9,115 of 24,918 incidents (36.6%) breached their SLA and that 94.2% are Moderate priority. Breach rates differ sharply by priority: Critical and High incidents breach almost always (98.1% and 99.5%), against 35.5% for Moderate and 15.9% for Low. Incidents reassigned at least once breached 54.4% of the time, against 21.6% for those never reassigned, and incidents that used a knowledge article breached 73.2% of the time, against 30.5% for those that did not. Knowledge use is most common for Low-priority incidents (30.2%), so the RQ3 association must be adjusted for priority. These are unadjusted descriptions, not hypothesis tests.

Table 6

Incident-Level Profile by Priority

Priority (final record)	Incidents	Share of incidents	SLA breach rate	Reassigned at least once	Knowledge base used
1 – Critical	270	1.1%	98.1%	58.5%	12.2%
2 – High	408	1.6%	99.5%	62.5%	19.4%
3 – Moderate	23,466	94.2%	35.5%	45.4%	13.7%
4 – Low	774	3.1%	15.9%	37.9%	30.2%
All	24,918	100%	36.6%	45.6%	14.3%

Note. Computed from the incident-level file (final record per incident). Data from Amaral et al. (2018b).

Analytic Approach

The analysis moves from description to inference to prediction. RQ1 to RQ3 use hypothesis tests with effect sizes, so that statistical significance, which is easy to reach with 24,918 observations, is always reported with practical magnitude. RQ4 uses supervised machine learning on a time-ordered hold-out set. Table 7 summarises each research question's method, justification, assumptions, and outputs; the subsections that follow give the solution to each research question with its evaluation metrics, formulae, and worked calculations. The worked numbers are illustrations of how each metric is computed, not results.

Table 7

Mapping of Research Questions to Analytic Methods

RQ	Method	Justification	Assumptions and checks	Outputs
RQ1	χ^2 test of independence on the 4×2 priority-by-breach table; pairwise 2×2 χ^2 tests with Holm correction	Binary outcome against a four-level categorical factor	Independent observations (one row per incident); all expected cell counts ≥ 5 (minimum 98.8); standardised residuals show which cells drive the result	χ^2 , p-value, Cramér's V, breach rate per priority with 95% CIs, adjusted pairwise p-values
RQ2	χ^2 test of independence; multivariable logistic regression with Wald test on β_1	Binary outcome; count predictor; need to adjust for priority and category	All expected cell counts ≥ 5 ; linearity in the logit (checked by refitting reassignment as 0, 1, 2, 3+); variance inflation factor < 5	χ^2 , ϕ ; adjusted odds ratio with 95% CI; McFadden pseudo- R^2
RQ3	χ^2 test of independence; multivariable logistic regression with Wald test on β_1 ; Mantel–Haenszel odds ratio across priority strata	Binary outcome and binary predictor; knowledge use is confounded with priority and category, so adjustment is required	All expected cell counts ≥ 5 ; variance inflation factor < 5 ; consistency of the odds ratio across priority strata checked	χ^2 , ϕ ; unadjusted and adjusted odds ratios with 95% CIs; McFadden pseudo- R^2

RQ	Method	Justification	Assumptions and checks	Outputs
RQ4	Priority-only logistic regression (baseline); regularised logistic regression; random forest (Breiman, 2001); XGBoost (Chen & Guestrin, 2016); DeLong test (DeLong et al., 1988)	Mixed and high-cardinality features with possible non-linear interactions; logistic regression gives an interpretable benchmark	Only at-logging features (no leakage); time-ordered 70/30 split (cut on 2 May 2016); 5-fold cross-validation for tuning; class weighting for 36.6% prevalence	AUC with 95% CI, DeLong p-value, precision, recall, F1, Brier score, permutation feature importance

Note. CI = confidence interval. Knowledge-base use is excluded from the main RQ4 model and added only in a sensitivity model.

Solution to RQ1: Priority and SLA Breach

Approach: statistical technique. H_{01} is tested with a χ^2 test of independence on the 4×2 table of priority by breach outcome. If H_{01} is rejected, standardised residuals show which priorities breach more or less often than expected, and pairwise 2×2 χ^2 tests with Holm correction identify which priorities differ. Breach rates for each priority are reported with 95% confidence intervals.

Evaluation metrics. The test statistic and its effect size are

$\chi^2 = \sum (O - E)^2 / E$, $E = (\text{row total} \times \text{column total}) / N$, Cramér's $V = \sqrt{(\chi^2 / (N \times (k - 1)))}$, where O and E are the observed and expected counts in each cell and $k = 2$ is the smaller table dimension. For example, the expected number of breaches among the 270 Critical incidents, if breach were independent of priority, is $270 \times 9,115 / 24,918 = 98.8$. With $N = 24,918$, a statistic of $\chi^2 = 2,492$ would give $V = \sqrt{(2,492 / 24,918)} = 0.32$, a medium effect against Cohen's (2013) thresholds of 0.10 (small), 0.30 (medium), and 0.50 (large).

Expected solution. The breach rate for each priority and whether the differences are significant, showing whether the priority scheme and its SLA targets work as intended.

Solution to RQ2: Reassignment and SLA Breach

Approach: statistical technique. H_{02} is first tested with a χ^2 test of independence on the 2×2 table of reassigned (yes/no) by breached (yes/no). The adjusted association is then

estimated with a multivariable logistic regression, $\text{logit}(P(\text{breach} = 1)) = \beta_0 + \beta_1 \times \text{reassignment_count} + \beta_2 \times \text{priority} + \beta_3 \times \text{category}$, and H_{02} is tested with the Wald test on β_1 . Linearity in the logit is checked by refitting reassignment as a category (0, 1, 2, 3+), and multicollinearity with variance inflation factors.

Evaluation metrics. The test statistic is $\chi^2 = \sum (O - E)^2 / E$ over the four cells, with association strength $\phi = \sqrt{(\chi^2 / N)}$. The adjusted effect is the odds ratio $OR = e^{\beta_1}$ with 95% confidence interval $e^{(\beta_1 \pm 1.96 \times SE(\beta_1))}$; for example, $\beta_1 = 0.40$ would give $OR = e^{0.40} = 1.49$, meaning each extra reassignment multiplies the odds of breach by 1.49. Model fit is McFadden's pseudo- $R^2 = 1 - \ln L(\text{model}) / \ln L(\text{null})$.

Expected solution. The breach odds attributable to each additional reassignment, which puts a number on the value of routing a ticket to the right group first time.

Solution to RQ3: Knowledge-Base Use and SLA Breach

Approach: statistical technique. H_{03} is first tested with a χ^2 test of independence on the 2×2 table of knowledge used (yes/no) by breached (yes/no), for the 3,551 incidents that used a knowledge article and the 21,367 that did not. Because knowledge use is unevenly spread across priorities and categories, the adjusted association is estimated with a multivariable logistic regression, $\text{logit}(P(\text{breach} = 1)) = \beta_0 + \beta_1 \times \text{knowledge} + \beta_2 \times \text{priority} + \beta_3 \times \text{category}$, and H_{03} is tested with the Wald test on β_1 . A Mantel–Haenszel odds ratio across the four priority strata (Mantel & Haenszel, 1959) is reported as a robustness check.

Evaluation metrics. The unadjusted odds ratio from the 2×2 table is $OR = (a \times d) / (b \times c)$, where a and b are breached and met incidents with knowledge use and c and d the same without it; the adjusted odds ratio is e^{β_1} with 95% confidence interval $e^{(\beta_1 \pm 1.96 \times SE(\beta_1))}$. For example, if 1,000 of 2,000 incidents with knowledge use breached against 5,000 of 20,000 without, the unadjusted odds ratio would be $(1,000 \times 15,000) / (1,000 \times 5,000) = 3.0$, meaning the odds of breach are three times higher with knowledge use.

Expected solution. Evidence on whether knowledge-base use is associated with breach once priority and category are held constant, which tells knowledge managers whether articles

are helping or are mainly consulted for incidents that were already at risk. Results are reported as association, not causation.

Solution to RQ4: Predicting SLA Breach at Logging

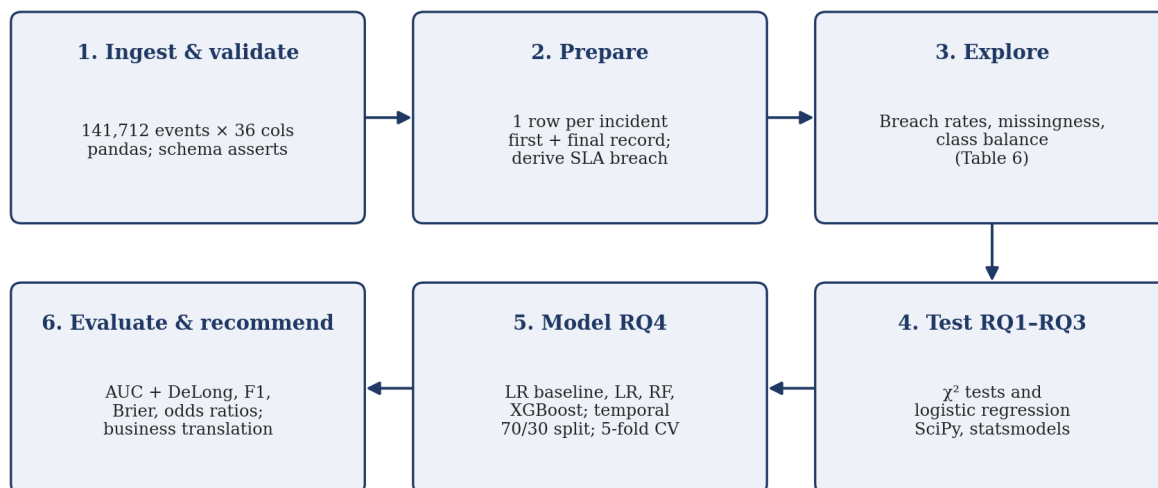
Approach: machine learning and statistical technique. Three classifiers are trained on attributes available at logging (Table 5): regularised logistic regression, random forest (Breiman, 2001), and XGBoost (Chen & Guestrin, 2016). The baseline is a logistic regression on priority alone, the rule operations teams already use. Incidents are ordered by opening time, and the earliest 70% form the training set and the latest 30% the test set; hyperparameters are tuned by five-fold cross-validation within the training set, and class weighting handles the 36.6% breach prevalence. H_{04} is tested statistically with the DeLong test for correlated ROC curves (DeLong et al., 1988), since both models score the same test incidents.

Evaluation metrics. With TP, FP, TN, and FN as the confusion-matrix counts: Precision = $TP / (TP + FP)$, Recall = $TP / (TP + FN)$, $F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$, $AUC = P(\text{score of a breached incident} > \text{score of a non-breached incident})$, and Brier score = $(1 / N) \times \sum (p_i - y_i)^2$. For example, a model that flags 1,000 test incidents, of which 700 breach, out of 2,000 actual breaches would have precision = $700 / 1,000 = 0.70$, recall = $700 / 2,000 = 0.35$, and $F1 = 2 \times 0.70 \times 0.35 / 1.05 = 0.47$. AUC is the primary metric because it does not depend on the threshold; recall is the key operational metric because a missed breach costs more than an unnecessary review.

Expected solution. A validated breach-risk score available at logging, evidence of whether it beats the priority-only rule, and the attributes that drive risk.

End-to-End Solution Pipeline

Figure 3 shows the pipeline from raw data to recommendation, and Table 8 lists the tools and the artefact each stage produces. Every stage is scripted in a numbered Jupyter notebook in the GitHub repository; random seeds are fixed (seed = 42), package versions are pinned in requirements.txt, and the raw file is never modified, so any reader can reproduce every table and figure by running the notebooks in order.

Figure 3*Solution Pipeline From Raw Event Log to Recommendation***Table 8***Pipeline Stages, Tools, and Outputs*

Stage	Tools	Output artefact
1–2. Ingest, validate, prepare	Python 3, pandas, NumPy	data/processed/incidents.csv; exclusion counts (01_data_preparation.ipynb)
3. Explore	pandas, matplotlib	Descriptive profile (Table 6) and distribution plots
4. Test RQ1–RQ3	SciPy, statsmodels	χ^2 statistics, odds ratios, p-values, effect sizes, confidence intervals (notebooks 02–04)
5. Model RQ4	scikit-learn, XGBoost	Trained baseline and candidate models; tuned hyperparameters (notebook 05)
6. Evaluate and recommend	scikit-learn metrics; DeLong test implemented in Python	AUC comparison, confusion matrices, feature importance, business translation for the final report

Note. Expected outcomes, all for SLA breach: breach rates by priority (RQ1), the breach odds attributable to reassignment (RQ2) and to knowledge-base use (RQ3), and a validated at-logging breach-risk score compared against the priority-only rule (RQ4).

Limitations and Threats to Validity

The data comes from one anonymised organisation with coded categories, which limits generalisability and the interpretation of individual levels. The three-month concentration of incidents prevents any study of seasonality. RQ2 and RQ3 are observational, so unmeasured factors such as analyst skill may confound the associations. SLA targets are not recorded; the near-universal breach of Critical and High incidents suggests very tight targets that cannot be verified. Priority is highly imbalanced, which limits precision for the three smaller groups. Each limitation will be discussed with the results.

Recommendation and Application

The primary target users are service delivery managers and service desk leads in managed-service providers and internal IT organisations, who are accountable for SLA attainment. Secondary users are ITSM platform owners, who configure routing and priority rules, and knowledge managers, who decide where to invest in knowledge articles. Table 9 shows how each result turns into a decision. The final report will express model performance in operational terms, such as the share of breaches caught per 100 tickets flagged, so that the recommendation can be judged on workload and benefit rather than on AUC alone.

If RQ4 is supported, the recommended implementation path is a one-month pilot in shadow mode, in which the model scores every new ticket but no routing changes, followed by a comparison of flagged tickets with actual breaches. The alert threshold would then be set to match the capacity of the senior triage team rather than chosen for statistical neatness, and the model would be retrained each quarter and monitored for drift in the breach rate and in the mix of categories. Because no analyst-level variables are used, the score rates tickets, not people, which keeps its use within normal service-governance practice.

Table 9*Stakeholders, Decisions, and Actions Supported by Each Research Question*

RQ	Stakeholder	Decision supported	Action if the alternative hypothesis is supported
RQ1	Service delivery manager; client	Whether the priority scheme and SLA targets work as intended	Review SLA targets and staffing for priorities with near-certain breach
RQ2	Service desk lead; ITSM platform owner	How much first-time-right routing is worth	Invest in routing rules and resolver-group skills matrices; track reassignment as a KPI
RQ3	Knowledge manager	Whether knowledge articles reduce breach risk or mainly signal harder incidents	Prioritise article quality for high-breach categories; track knowledge coverage
RQ4	Service desk lead; service delivery manager	Which new tickets to escalate early	Embed the risk score at logging and route high-risk tickets to senior triage

Project Plan and Resources

The project needs no budget: the data is free, and all analysis uses open-source Python libraries on a standard laptop, where the 141,712-row file loads in seconds. Table 10 sets out a phased 10-week plan aligned with the QM640 deadlines; data preparation is already complete, which leaves the most time for the inferential tests and the RQ4 modelling.

Table 10*Phased Work Plan Aligned to the QM640 Schedule*

Week (due date)	Phase	Deliverable	Analytical work and tools
1–2 (6 Oct 2026)	Define	Synopsis draft	Topic approval; data validated; GitHub repository and preparation notebook published (pandas)
3 (13 Oct 2026)	Define	Synopsis final	Mentor feedback incorporated; literature search
4–5 (20 and 27 Oct 2026)	Analyse	Interim report draft and final	RQ1–RQ3 χ^2 tests and logistic regressions (SciPy, statsmodels)
6 (3 Nov 2026)	Model	Final report draft	RQ4 baseline and candidate models, tuning, DeLong test (scikit-learn, XGBoost)
7–8 (15 and 22 Nov 2026)	Validate	Mentor and faculty reviews	Sensitivity analyses; interpretation; business translation
9 (29 Nov 2026)	Communicate	Final presentation	Presentation deck and model demonstration
10 (13 Dec 2026)	Communicate	Final report	Final documentation, references, and repository clean-up

Note. Dates are taken from the QM640 schedule.

References

- Amaral, C. A. L., Fantinato, M., & Peres, S. M. (2018a). Attribute selection with filter and wrapper: An application on incident management process. In *Proceedings of the 2018 Federated Conference on Computer Science and Information Systems* (Annals of Computer Science and Information Systems, Vol. 15, pp. 679–682).
<https://doi.org/10.15439/2018F126>
- Amaral, C., Fantinato, M., & Peres, S. (2018b). *Incident management process enriched event log* [Data set]. UCI Machine Learning Repository. <https://doi.org/10.24432/C57S4H>
- Amaral, C. A. L., Fantinato, M., Reijers, H. A., & Peres, S. M. (2019). Enhancing completion time prediction through attribute selection. In *Information technology for management: Emerging research and applications* (Lecture Notes in Business Information Processing, pp. 3–23). Springer. https://doi.org/10.1007/978-3-030-15154-6_1
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Cohen, J. (2013). *Statistical power analysis for the behavioral sciences* (2nd ed.). Routledge.
<https://doi.org/10.4324/9780203771587>
- Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (pp. 4–5). IEEE. <https://doi.org/10.1109/ICSE-Companion.2019.00023>
- DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3), 837–845. <https://doi.org/10.2307/2531595>

- Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods*, 39(2), 175–191. <https://doi.org/10.3758/BF03193146>
- Gelman, A., & Hill, J. (2007). *Data analysis using regression and multilevel/hierarchical models*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790942>
- Green, S. B. (1991). How many subjects does it take to do a regression analysis? *Multivariate Behavioral Research*, 26(3), 499–510. https://doi.org/10.1207/s15327906mbr2603_7
- Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36. <https://doi.org/10.1148/radiology.143.1.7063747>
- Mantel, N., & Haenszel, W. (1959). Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute*, 22(4), 719–748. <https://doi.org/10.1093/jnci/22.4.719>
- Marrone, M., & Kolbe, L. M. (2011). Impact of IT service management frameworks on the IT organization: An empirical study on benefits, challenges, and processes. *Business & Information Systems Engineering*, 3(1), 5–18. <https://doi.org/10.1007/s12599-010-0141-5>
- Shao, Q., Chen, Y., Tao, S., Yan, X., & Anerousis, N. (2008). Efficient ticket routing by resolution sequence mining. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 605–613). ACM. <https://doi.org/10.1145/1401890.1401964>
- Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive business process monitoring with LSTM neural networks. In E. Dubois & K. Pohl (Eds.), *Advanced information systems engineering* (Lecture Notes in Computer Science, Vol. 10253, pp. 477–492). Springer. https://doi.org/10.1007/978-3-319-59536-8_30
- van der Aalst, W. (2016). *Process mining: Data science in action* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-49851-4>

Verenich, I., Dumas, M., La Rosa, M., Maggi, F. M., & Teinmaa, I. (2019). Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring. *ACM Transactions on Intelligent Systems and Technology*, 10(4), 1–34.
<https://doi.org/10.1145/3331449>

Appendix A

Sample Size Calculation Code

The code below, saved as `src/sample_size.py` in the GitHub repository, reproduces Tables

3 and 4 (Hanley & McNeil, 1982, for RQ4).

```
# QM640 capstone - minimum sample sizes (alpha = 0.05, power = 0.80)
# Dependent variable for all RQs: SLA breach (binary)
import numpy as np
from scipy.stats import chi2, ncx2

def chi2_n(w, df, a=0.05, p=0.80):          # chi-square test of independence
    n, c = 10, chi2.ppf(1 - a, df)
    while ncx2.sf(c, df, n * w**2) < p:
        n += 1
    return n

def auc_test_n(auc, e, prev):              # RQ4 (Hanley & McNeil, 1982)
    q1, q2 = auc / (2 - auc), 2 * auc**2 / (1 + auc)
    n = 50
    while True:
        pos = round(n * prev); neg = n - pos
        var = (auc*(1-auc) + (pos-1)*(q1-auc**2) + (neg-1)*(q2-auc**2)) / (pos*neg)
        if 1.96 * np.sqrt(var) <= e:
            return n
        n += 1

rq1 = chi2_n(0.10, 3)                      # priority (4) x breach (2): 1091
rq2 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.047))) # 785 / 0.953 -> 824
rq3 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.154))) # 785 / 0.846 -> 928
test = auc_test_n(0.75, 0.03, 9115 / 24918) # observed prevalence 0.366: 1191
rq4 = int(np.ceil(test / 0.30))            # 3970
print(rq1, rq2, rq3, test, rq4, "final N =", max(rq1, rq2, rq3, rq4))
```